Payroll System Architecture and Design

- Core Logic and Functional Requirements 💰
  - Calculates total salaries, performance bonuses, and tax deductions.
  - Manages employee data through structured records.
  - Generates financial outputs based on predefined business rules.
- Role of Abstract Data Types (ADT) 🏗️
  - Defines the logical properties and operations of payroll components.
  - Establishes a formal contract for how data interacts with functions.
  - Serves as the foundation for the system's data hierarchy.
  - Concept of Data Abstraction
    - Hides implementation details to focus on high-level operations.
    - Enables developers to use components without knowing their internal code.
    - Separation of Concerns — Distinguishes the interface's definition from its actual coding logic.
- Implementation and System Execution 💻
  - Translates abstract models into executable software modules.
  - Maps functions to specific payroll processing tasks.
  - Ensures that data abstraction is maintained during the coding phase.
- Reusability and Long-term Maintainability 🔄
  - Allows standardized payroll logic to be used across multiple departments.
  - Simplifies system updates by isolating changes to specific code segments.
  - Minimizes errors during the software lifecycle by ensuring code stability.
- Advantages of Modular Design 🧩
  - Breaks complex payroll calculations into smaller, independent units.
  - Facilitates parallel development by allowing multiple teams to work at once.
  - Improves debugging efficiency by isolating faults within individual modules.
- Justification for ADT Integration ⚖️
  - Protects data integrity by preventing direct access to sensitive records.
  - Enhances flexibility, allowing implementation changes without affecting the user.
  - Promotes 'don't repeat yourself' principles in system architecture.